Full-text search

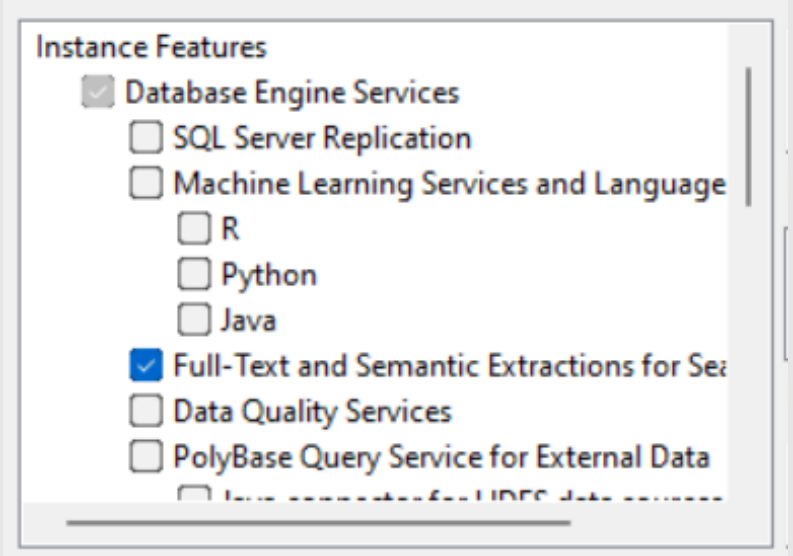
Baza danych: SQL Server

Przemysław Starosta - 2026

Wstęp

Wyszukiwanie pełnotekstowe to opcjonalny składnik bazy danych programu SQL Server.

Jeśli podczas instalacji programu SQL Server nie wybrano opcji Wyszukiwanie pełnotekstowe, uruchom ponownie Instalatora programu SQL Server, aby go dodać.



Wstęp

Usługa pełnego tekstu (fdhost.exe) może przeszukiwać ogromne ilości danych przy użyciu specjalnej składni FULL TEXT SEARCH, szukania form słów, fraz, wyszukiwanie tezaurusów czy bliskość słów.

Indeksy pełnotekstowe są tworzone i używane w kontekście danego języka, takiego jak angielski lub francuski. A zapytania pełnotekstowe wykonują wyszukiwanie słów lub fraz w oparciu o reguły tego konkretnego języka

Mechanizm dostępny już w SQL Server Express

Wstęp

W zapytaniach pełnotekstowych nie jest rozróżniana wielkość liter. Na przykład wyszukiwanie hasła „Aluminium” lub „aluminium” zwraca te same wyniki.

Zapytania pełnotekstowe używają niewielkiego zestawu predykatów Transact-SQL (**CONTAINS** i **FREETEXT**) oraz ich odpowiedniki w funkcji (**CONTAINSTABLE** i **FREETEXTTABLE**). Funkcje tabelaryczne zwracają wartość zrankingowaną.

Indeks

Zgodnie z projektem indeksy pełnotekstowe wymagają unikatowego nieklastrowanego lub klastrowanego indeksu wierszy w pliku tabeli, w której są tworzone, z pojedynczą kolumną w kluczu.

Polecam taki indeks mieć klucz typu całkowitego ze względu na wydajność, na przykład kolumna *identity*.

Indeksy pełnotekstowe są zwykle umieszczane na kolumnach varchar lub nvarchar, często o dużej długości, ale możesz też umieść je w kolumnach xml i varbinary.

Sprawdzenie, czy zainstalowany

```
SELECT FULLTEXTSERVICEPROPERTY('IsFullTextInstalled')
AS FULLTEXTSERVICE
```

	FULLTEXTSERVICE
1	1

Tworzenie katalogu i indeksu

```
create fulltext catalog FullTextCatalog as default

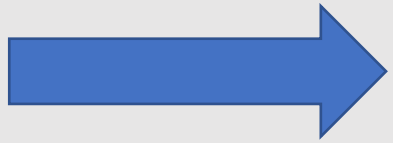
create fulltext index on Employees(last_name)
key index emp_emp_id_pk
```

UWAGA: key index to wskazania nazwy istniejącego INDEKSU unikalnego na którym oprzemy indeks tekstowy

W jednej tabeli może być tylko JEDEN indeks tekstowy

Tworzenie indeksu ze wskazaniem języka

```
create fulltext index on Pracownicy
(
        Opis Language Polish
)
key index pracPK
CHANGE_TRACKING = AUTO –auto | manual | off
STOPLIST = SYSTEM
```



Aktualizacja indeksu pełnotekstowego

Określa, czy zmiany (aktualizacje, usunięcia lub wstawienia) wprowadzone do kolumn tabeli, które są objęte indeksem pełnotekstowym, będą propagowane przez SQL Server do indeksu pełnotekstowego. Zmiany danych za pomocą WRITETEXT i UPDATETEXT nie są odzwierciedlane w indeksie pełnotekstowym i nie są zbierane przy śledzeniu zmian.

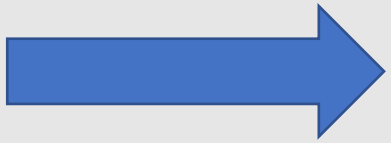
- **MANUAL** - Określa, że śledzone zmiany muszą być propagowane ręcznie przez wywołanie instrukcji ALTER FULLTEXT INDEX... START UPDATE POPULATION Transact-SQL instrukcja (ręczne wypełnianie). Możesz użyć programu SQL Server Agent do okresowego wywoływania tej instrukcji języka Transact-SQL.
- **AUTO** - Określa, że śledzone zmiany będą propagowane automatycznie w miarę modyfikowania danych w tabeli podstawowej (wypełnianie automatyczne). Chociaż zmiany są propagowane automatycznie, zmiany te mogą nie zostać natychmiast odzwierciedlone w indeksie pełnotekstowym. **AUTO jest ustawieniem domyślnym.**
- **OFF** - Określa, że SQL Server nie przechowuje listy zmian w indeksowanych danych. Jeśli nie określono opcji NO POPULATION, SQL Server zapełni indeks w pełni po jego utworzeniu. Opcji NO POPULATION można użyć tylko wtedy, gdy CHANGE_TRACKING jest wyłączone. Gdy określono NO POPULATION, SQL Server nie wypełnia indeksu po jego utworzeniu. Indeks jest wypełniany dopiero po wykonaniu przez użytkownika polecenia ALTER FULLTEXT INDEX z klauzulą START FULL POPULATION lub START INCREMENTAL POPULATION.

Stoplist

STOPLIST [=] { OFF | SYSTEM | stoplist_name }

Kojarzy listę zatrzymania pełnego tekstu z indeksem. Indeks nie jest wypełniony żadnymi tokenami, które są częścią określonej listy stopów. Jeśli nie określono STOPLIST, SQL Server kojarzy systemową listę stopów z pełnym tekstem z indeksem.

- **OFF** - Określa, że żadna lista stopów nie jest skojarzona z indeksem pełnotekstowym.
- **SYSTEM** - Określa, że dla tego indeksu pełnotekstowego powinien być używany domyślny system pełnotekstowy STOPLIST.
- **nazwa_listy_stopów** - Określa nazwę stoplist, która ma być skojarzona z indeksem pełnotekstowym.



Stoplist Stopwords

1. Stoplist – może zawierać wiele stopwords
2. Pozwala zapobiegać indeksowaniu (wyszukiwaniu)
3. Przykłady:
   - Łączniki: i, oraz, lub
   - Słowa często występujące w jakiejś dziedzinie

```
select * from sys.fulltext_stopwords
select * from sys.fulltext_stoplists
```

Wstęp

	CONTAINS	FREETEXT
Formy fleksyjne wyrazów	Na żądanie (FORMSOF(INFLECTIONAL, wyraz)	Zawsze
Thesaurus (też synonimy)	Na żądanie (FORMSOF(INFLECTIONAL, wyraz)	Zawsze
Wagi dla wyrazów	Tak	Nie
Wyrazy blisko siebie	Tak	Nie
Operatory logiczne	Tak	Nie
Przedrostki wyrazów(prefix)	Tak	Nie
Zapytanie	Złożone, większa kontrola	Dużo prostsze, mniejsza kontrola
Razem rezultaty	Mniejsza liczba wyników, dokładniejsze	Większa liczba wyników, mniej dokładne

Obsługa plików pdf

Możesz pobrać Adobe PDF iFilter 9 (64-bit) z:

ftp://ftp.adobe.com/pub/adobe/acrobat/win/9.x/PDFiFilter64installer.zip

Synonimy i plik tezaurus

1. Przechowywane na dysku
   - MSSQL\FTDATA tsXXX.xml (XXX – PLN, ENU)
2. Tsglobal – globalny tezaurus – wykorzystywany dodatkowo
3. Edycja pliku konfiguruje:
   - Diacritics_sensitive
   - Expansion (autor, pisarz) – wyszukiwane słowa bliskoznaczne
   - Replacement – wyszukiwane słowo jest zastępowanym słowem bliskoznacznym
   - Po edycji należy załadować go przez SQL Server

```
exec sys.sp_fulltext_load_thesaurus_file 1033
```

Stworzenie indeksu FTS

```
SELECT *
FROM employees
WHERE CONTAINS(last_name, '"en„')
```

Messages

```
Msg 7601, Level 16, State 2, Line 3
Cannot use a CONTAINS or FREETEXT predicate on table or indexed view 'employees' because it is not full-text indexed.
```

```
select *
  from sys.fulltext_catalogs

create fulltext catalog FullTextCatalog as default

select * from sys.fulltext_catalogs
```

Results | Messages

	fulltext_catalog_id	name	path	is_default	is_accent_sensitivity_on	data_space_id	file_id	principal_id	is_importing
1	5	FullTextCatalog	NULL	1	1	NULL	NULL	1	0

```
create fulltext index on Employees(last_name)
key index emp_emp_id_pk
```

Korzystanie z indeksu - CONTAINS

Wyszukuje precyzyjne lub rozmyte (mniej precyzyjne) dopasowania do pojedynczych słów i fraz, słów znajdujących się w pewnej odległości od siebie lub dopasowań ważonych w SQL Server.

CONTAINS może wyszukiwać:

- Słowo lub fraza.
- Przedrostek słowa lub frazy.
- Słowo obok innego słowa.
- Słowo generowane fleksyjnie z innego (na przykład słowo popęd jest rdzeniem fleksyjnym popędów, kierowanych, prowadzących i napędzanych).
- Słowo, które jest synonimem innego słowa używającego tezaurusa (na przykład słowo „metal” może mieć synonimy, takie jak „aluminium” i „stal”).

CONTAINS - przykłady(1/2)

- **SIMPLE TERM** – zapytanie zwraca rekord z dokładnie porównanym słowem King w kolumnie first_name i last_name

```
SELECT * FROM EMPLOYEES
WHERE
CONTAINS ((FIRST_NAME ,LAST_NAME), 'King')
```

- **PREFIX TERM** – zapytanie zwraca rekord rozpoczynające się na „King” w kolumnie i last_name

```
SELECT * FROM EMPLOYEES
WHERE
CONTAINS ((LAST_NAME), 'King*')
```

- **PROXIMITY TERM** – spowoduje wyszukanie Steven w ciągu 2 wyrazów King, ale tylko gdy Steven poprzedza King np. Steven Albert King

```
SELECT * FROM EMPLOYEES
WHERE
CONTAINS ((DESCRIPTION), 'NEAR (Steven, King), 10)')
```

CONTAINS - przykłady(2/2)

- **GENEARTION TERM** – zapytanie zwraca formy bliskoznaczne: Start, started, starting

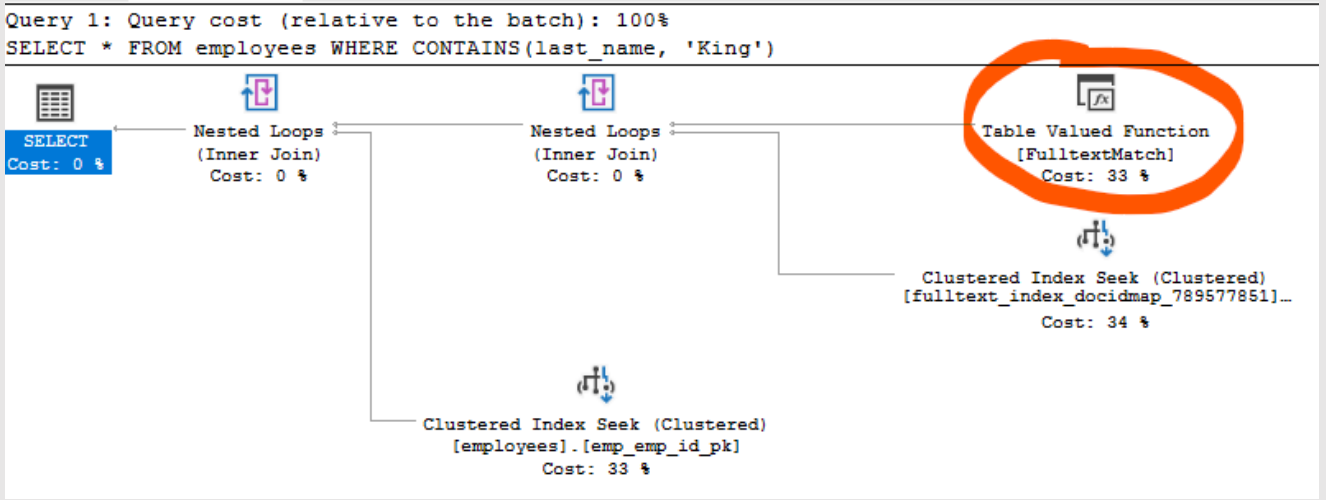
```
SELECT * FROM EMPLOYEES
WHERE
CONTAINS (*, 'FORMSOF (INFLECTIONAL, Start)')
```

- **THESAURUS TERM** – zapytanie zwraca synonimy dla słowa Start

```
SELECT * FROM EMPLOYEES
WHERE
CONTAINS (*, 'FORMSOF (THESAURUS, Start)')
```

Korzystanie z indeksu - CONTAINS

```sql
SELECT *
FROM employees
WHERE CONTAINS(last_name, 'King') -- tylko King
```

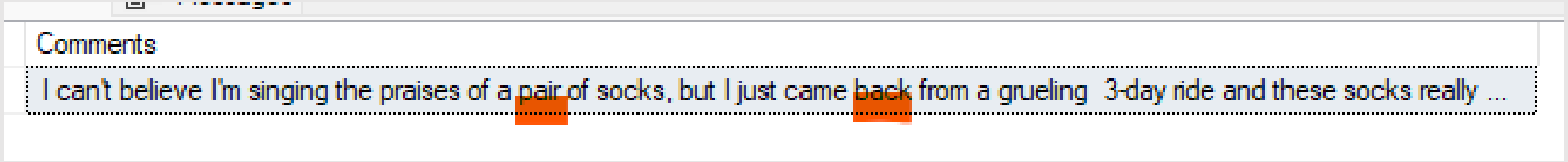



```sql
SELECT *
FROM employees
WHERE CONTAINS(last_name, 'King OR Ernst') -- King LUB Erst
```

Korzystanie z indeksu - CONTAINS

```
SELECT *
FROM employees
WHERE CONTAINS(last_name,'"K*" OR "D*"') – zaczynające się na „K” lub „D”
```

```
SELECT Comments
FROM Production.ProductReview
WHERE CONTAINS(Comments , 'NEAR((pair, back), 6, TRUE)'); -- przeszukuje tabelę Production.ProductReview pod kątem wszystkich komentarzy, które zawierają słowo pair w ciągu 6 wyrazów od słowa „back" i w określonej kolejności
```



Korzystanie z indeksu - CONTAINS

```
SELECT *
FROM employees
WHERE CONTAINS(last_name, '"K*"') --zaczynają się na „K", cudzysłowia należy również użyć przy frazach ze spacją
```

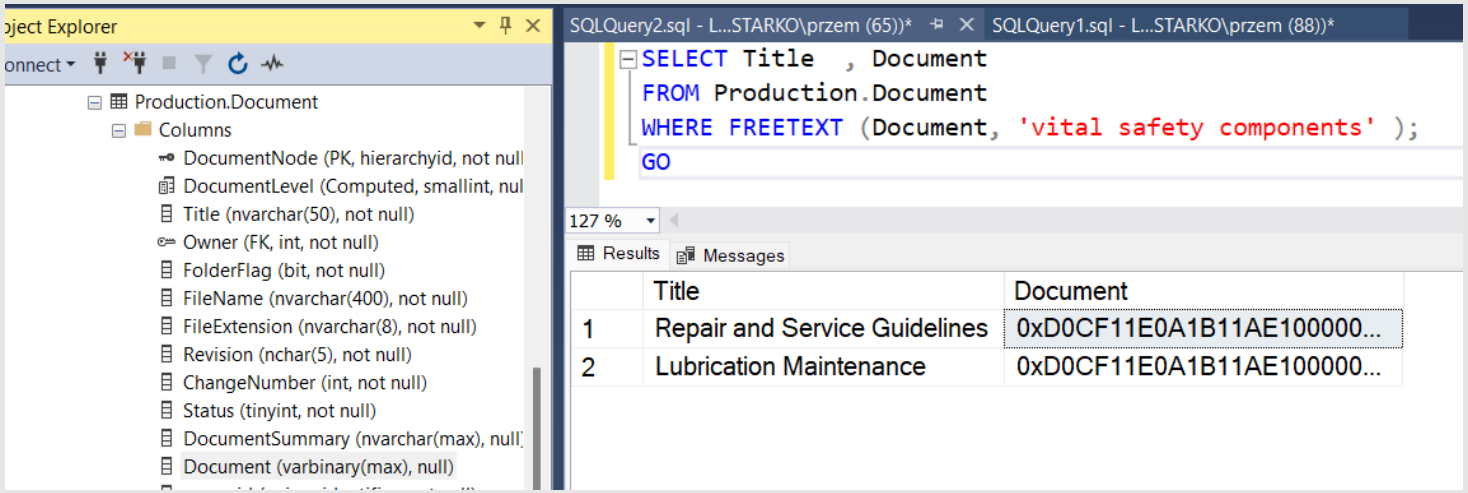
```
SELECT Description
FROM Production.ProductDescription
WHERE CONTAINS(Description, ' FORMSOF (INFLECTIONAL, ride) ');-szuka formy słowa ride: riding, ridden etc.
```

Korzystanie z indeksu - FREETEXT

Ten predykat wyszukuje wartości pasujące do znaczenia, nie tylko dokładne sformułowanie słów w warunku wyszukiwania, ale również znaczeniowo.

Gdy jest używany FREETEXT, pełnotekstowy aparat zapytań wewnętrznie wykonuje następujące akcje: przypisuje każdemu terminowi wagę, a następnie znajduje dopasowania.

Przydaje się również do przeszukiwania kolumn binarnych:



Korzystanie z indeksu - FREETEXT

Jednym z przypadków, w których uznałem to za przydatne (i jestem pewien, że może być więcej przypadków), jest potrzeba przeprowadzenia wyszukiwania fleksyjnego, tezaurusa i NEAR razem w jednym zapytaniu. Poniżej znajduje się przykład

```
SELECT * FROM EMPLOYEES
WHERE
FREETEXT ((DESCRIPTION), 'NEAR (Start, Teach), 10)’)
```

Powyższe zapytanie wyszuka wszystkie formy słowa, takie jak Start, Started, Teach, Taught, Teaching itp.

Ten rodzaj wyszukiwania nie jest łatwy do wykonania przy użyciu CONTAINS, ponieważ łączenie FORMSOF i NEAR nie jest dozwolone w jednym zapytaniu.

Korzystanie z indeksu - FREETEXT

```
DECLARE @SearchWord NVARCHAR(30);
SET @SearchWord = N'high-performance';
SELECT Description
FROM Production.ProductDescription
WHERE FREETEXT(Description, @SearchWord);
```

Results | Messages

	Description
1	This bike delivers a high-level of performance on a budget. It is responsive and maneuverable, and offers peace-of-mind whe
2	Top-of-the-line competition mountain bike. Performance-enhancing options include the innovative HL Frame, super-smooth fr
3	Travel in style and comfort. Designed for maximum comfort and safety. Wide gear range takes on all hills. High-tech aluminum
4	Superior shifting performance.
5	High-strength crank arm.
6	High-performance carbon road fork with curved legs.

FREETEXT ocenia nie tylko KONKRETNY zapis, ale szuka go w całości tekst poszukują *znaczenia*. Tutaj wynikiem są 22 wiersze

```
DECLARE @SearchWord NVARCHAR(30);
SET @SearchWord = N'high-performance';
SELECT Description
FROM Production.ProductDescription
WHERE CONTAINS(Description, @SearchWord);
```

Results | Messages

	Description
1	High-performance carbon road fork with curved le...
2	High-performance mountain replacement wheel.
3	High-performance mountain replacement wheel.

CONTAINS ocenia tylko KONKRETNY zapis, ale szuka go w całości tekst poszukują *znaczenia*. Tutaj wynikiem są 3 wiersze

Dlaczego używać?

- Wyszukiwanie pełnotekstowe to bardziej zaawansowany sposób przeszukiwania bazy danych. Wyszukiwanie pełnotekstowe szybko znajduje wszystkie wystąpienia terminu (słowa) w tabeli bez konieczności przeglądania wierszy i bez konieczności sprawdzania, w której kolumnie termin jest przechowywany.

- Wyszukiwanie pełnotekstowe działa przy użyciu indeksów tekstowych. Indeks tekstowy przechowuje informacje o pozycji dla wszystkich terminów znalezionych w kolumnach, na podstawie których tworzony jest indeks tekstowy.

- Używanie indeksu tekstowego może być szybsze niż używanie zwykłego indeksu do znajdowania wierszy zawierających daną wartość.

Dlaczego używać?

- Możliwość wyszukiwania pełnotekstowego w SQL Anywhere różni się od wyszukiwania przy użyciu predykatów, takich jak LIKE, REGEXP i SIMILAR TO, ponieważ dopasowywanie jest oparte na terminach, a nie na wzorcach.

- Porównywanie ciągów znaków w wyszukiwaniu pełnotekstowym używa wszystkich normalnych ustawień sortowania dla bazy danych. Na przykład, jeśli baza danych jest skonfigurowana tak, aby wielkość liter nie była rozróżniana, wówczas wyszukiwanie pełnotekstowe nie będzie uwzględniać wielkości liter.

- Wyszukiwanie pełnotekstowe ma zastosowanie w wielu scenariuszach biznesowych, takich jak e-biznes — wyszukiwanie elementów w witrynie internetowej; kancelarie prawne – wyszukiwanie historii spraw w repozytorium danych prawniczych; lub działy kadr — dopasowując opisy stanowisk do przechowywanych życiorysów

FTS vs LIKE

- W przeciwieństwie do wyszukiwania pełnotekstowego predykat LIKE działa tylko na wzorcach znaków.
- Nie można używać predykatu LIKE do wysyłania zapytań do sformatowanych danych binarnych.
- Zapytanie LIKE dotyczące dużej ilości nieustrukturyzowanych danych tekstowych jest znacznie wolniej niż równoważne zapytanie pełnotekstowe dotyczące tych samych danych.
- Zapytanie LIKE dotyczące milionów wierszy danych tekstowych może zająć kilka minut, podczas gdy zapytanie pełnotekstowe może zająć tylko sekundy lub mniej w odniesieniu do tych samych danych, w zależności od liczby zwracanych wierszy”.
- Tak więc główna różnica polega na rodzaju wyszukiwania, które wykonują: pełny tekst dotyczy wyszukiwania słów, podczas gdy LIKE dotyczy wyszukiwania znaków. Osobiście użyłbym wyszukiwania pełnotekstowego w tabelach, w których mam duże ilości tekstu ludzkiego - zwykle duże kolumny (duże opisy, przekraczające standardowe 8 000 znaków) w dużych tabelach (miliony wierszy).